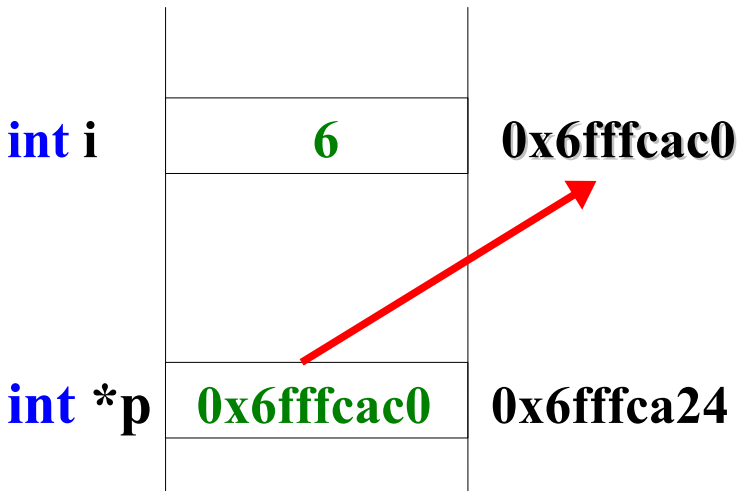
# Der Datentyp Zeiger

# Grundsätzliches: Was ist ein Zeiger

- Datentyp dessen Werte für Adressen von Speicherzellen stehen
- Variable vom Datentyp Zeiger
  - steht für eine Adresse irgendwo im Speicher
  - hat einen Wert
  - Wert = Adresse einer anderen Variablen
  - hat eine festgelegte Größe




- ~~zur erhöhten Sicherheit wird Verwendung durch Standards wie ANSI C, Standard-C99, stark eingeschränkt~~

Achtung: ~~Moderne Programmiersprachen (wie etwa Java) verzichten aus Sicherheitsgründen vollständig auf Zeiger~~

# Deklaration von Zeigern

- zu jedem bekannten Datentyp T gibt es einen Zeiger-Typ T*
- allgemeine Syntax zur Deklaration von Zeigern:

```
DatenTyp *variablenName [= Wert];
```

```
int i = 42;
int *p1, j;        // j ist kein Zeiger auf int
                   // sondern eine Variable vom Typ int
int *p2, *p3;      // p1, p2 und p3 sind Zeiger auf int
```

# Eigenschaften von Zeigern

- eine Zeigervariable ist immer gleich groß, unabhängig von Größe des Typs auf den der Zeiger verweist (Wortbreite der CPU-Architektur)
- Deklaration eines Zeigers *T** erzeugt nur Speicherplatz zum Ablegen der Adresse, nicht für das *Objekt vom Typ T*
- haben Eigenschaften normaler Datentypen
  - Arithmetik (hier: Adressenberechnungen)
  - Zuweisung, Vergleiche
- **Vorteil: Direkter Zugriff auf den Speicher des Computers**
  - **unverzichtbar zur schnellen, maschinennahen Programmierung**
- Nachteil:
  - keine Überprüfung auf korrekte Handhabung möglich
  - Eigenschaft der "strengen Typisierung" wird aufgehoben

# Adressoperator &

- Abfragen der Adresse einer Variablen mit Adressoperator &

  pvar = &var;

  wobei var vom Typ T und pvar vom Typ T* ist

```
int *pi
int i = 42;
pi =&i;

float f;
float *pf = &f;
float **ppf = &pf;      // ppf enthält Adresse auf float-Zeiger
```

Achtung: ohne Typumformungen sind in C nur Zuweisungen von Zeigern mit gleichen Typ oder void-Zeigern erlaubt

# Deferenzierungsoperator *

- Operator * wird zum Zugriff auf den Inhalt des Objekts, auf den ein Zeiger zeigt, verwendet

  T var = *ptr-expr;

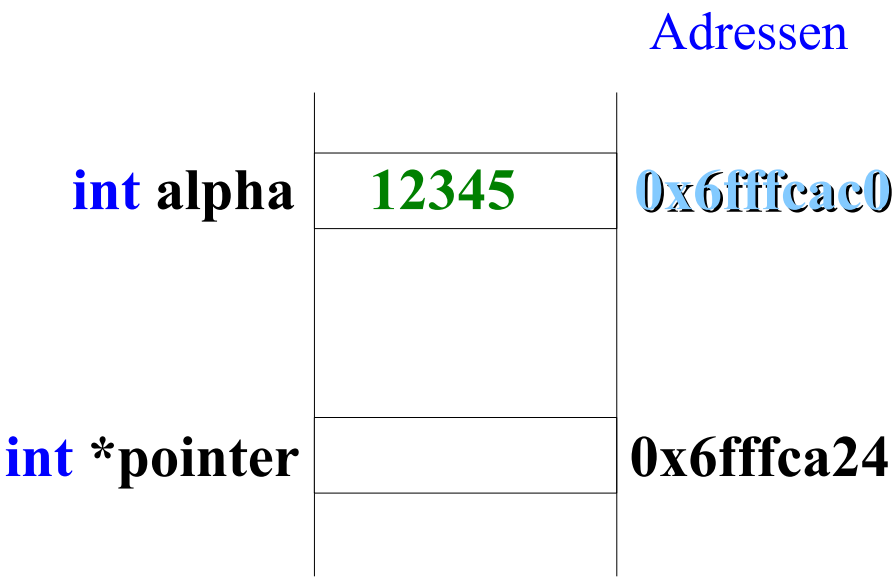
  wobei ptr-expr für einen Ausdruck vom Typ T* steht.
  Resultatswert ist dann vom Typ T

- unärer Operator * heißt Dereferenzierungsoperator

Achtung: Nicht zu verwechseln mit dem Modifizierer * zur Deklaration eines Zeigers!

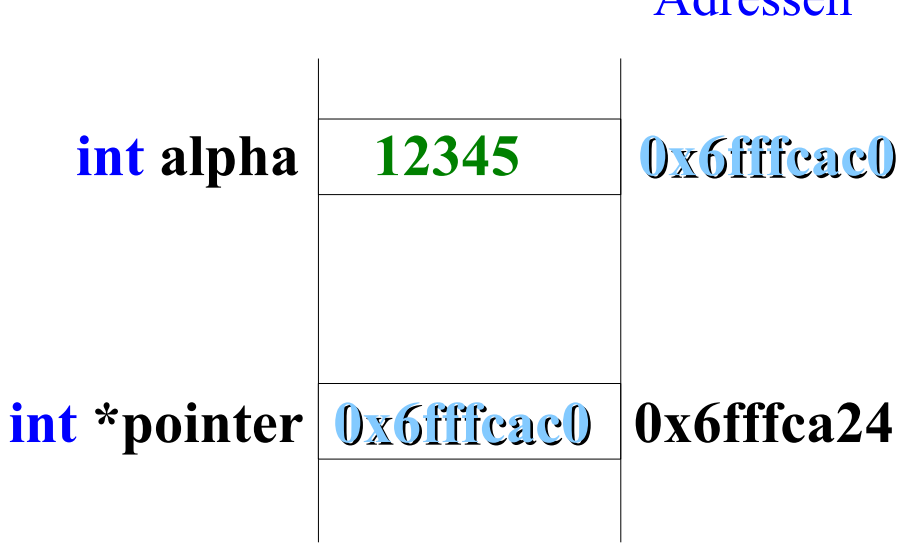
# Beispiel: Verwendung des Dereferenzierungsoperators

```
int main(void){
      int alpha;
      int *pointer;
      alpha = 12345;
      pointer  = &alpha;
      *pointer = 67890;
      return 0;
}
```



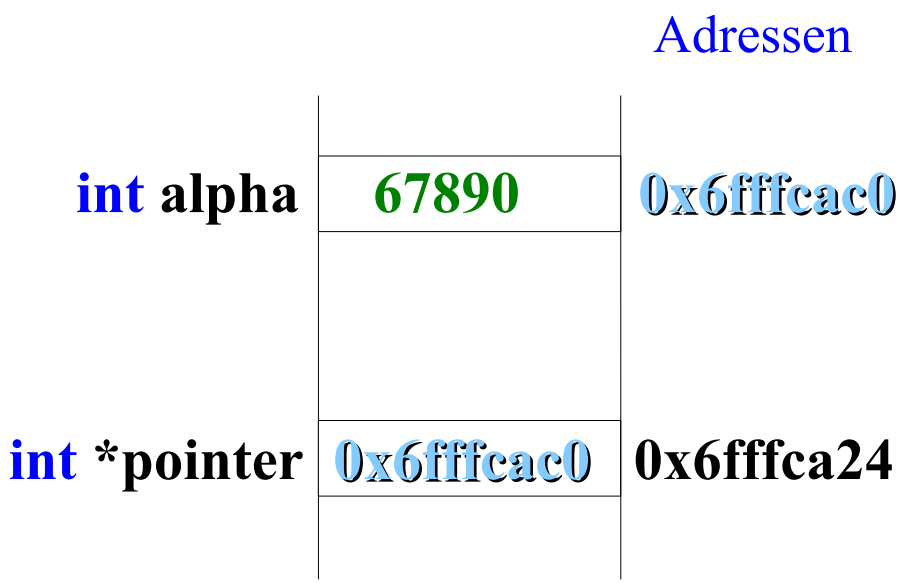
# Beispiel: Verwendung des Dereferenzierungsoperators

```
int main(void){
      int alpha;
      int *pointer;
      alpha = 12345;
      pointer  = &alpha;
      *pointer = 67890;
      return 0;
}
```



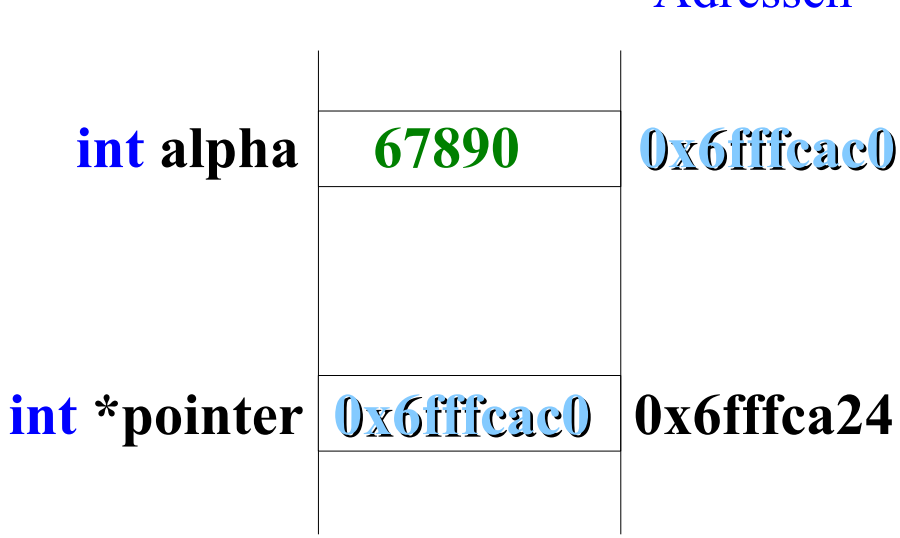
# Beispiel: Verwendung des Dereferenzierungsoperators

```
int main(void){
      int alpha;
      int *pointer;
      alpha = 12345;
      pointer  = &alpha;
      *pointer = 67890;
      return 0;
}
```



# Beispiel: Verwendung des Dereferenzierungsoperators

```
int main(void){
      int alpha;
      int *pointer;
      alpha = 12345;
      pointer  = &alpha;
      *pointer = 67890;
      return 0;
}
```




Achtung: * ist die Umkehrfunktion zu &
Es gilt: *&alpha ist äquivalent zu alpha,
aber auch *&*&*&*&alpha ist äquivalent zu alpha

# Beispiel: Ausgabe einer Adresse

```
#include <stdio.h>

int main(void){
      int alpha;
      int *pointer;
      alpha = 12345;
      pointer  = &alpha;
      *pointer = 67890;

      printf("Ausgabe des Werts %d \n", *pointer);
      printf("Ausgabe der Adresse %p \n", pointer);
      printf("Ausgabe des Werts %d \n", alpha);
      printf("Ausgabe der Adresse %p \n", &alpha);
      return 0;
}
```

Adressenausgabe mit Formatzeichen %p

# Zeiger auf void

- ist der Typ, auf den ein Zeiger zeigen soll, nicht bekannt, kann ein Zeiger auf Typ void vereinbart werden

```
void *vp;
```

- ein Zeiger auf void ist ein untypisierter Zeiger, der zu allen anderen Zeigern kompatibel ist

```
int *ip; float *fp;
    ...
vp = ip;
fp = vp; // erlaubte Anweisungen
```

- abgesehen von void-Zeigern dürfen ohne Typkonvertierung in C keine verschiedenartigen Zeigertypen zugewiesen werden

```
ip = fp; // Fehler, nur mit Typumwandlung (int *) erlaubt
```

- Achtung: da void-Zeiger auf Objekte unbekannter Art zeigen, dürfen (besser: können) diese nicht dereferenziert werden

# Nullzeiger

- Konstante NULL ist in C eine ungültige Adresse, die in <stddef.h> als (void *) 0 definiert ist

```
int *pointer;
pointer = NULL;          // NULL aus <stddef.h>
pointer = (void *) 0;    // äquivalent zu oben
```

Bemerkung: C-Programme legen in Adresse 0 per Konvention niemals ein Objekt ab

- Dereferenzierung des Nullzeigers führt zu einem
  - Speicherzugriffsfehler (führt zum Abbruch des Programms)
  - „segmentation fault“
  - passiert auch (aber nicht immer) bei nicht initialisierten Zeigern
  - Aufgabe des Betriebssystems („paging“)

# Vergleichsoperatoren für Zeiger

- Zeiger kann man auf Gleichheit (==) und Ungleichheit (!=) hin überprüfen
  - Gleichheit bedeutet: zeigt auf dieselbe Variable (d.h. Inhalt ist dieselbe Speicheradresse) oder beide Operanden sind NULL
- alle anderen Vergleiche wie < , > , <= , >=, etc. sind erlaubt
  - Für gewöhnliche Programme eher ohne Anwendung
  - Aber stellen Sie sich vor, Sie wollen ein Betriebssystem implementieren

# Explizite Speicheranforderung und -freigabe

- in C kann Programmierer zur Laufzeit explizit dynamisch Speicherplatz anfordern bzw. freigeben
  - direktes Einrichten eines neuen Datenobjektes mit malloc() oder calloc()

```
#include <stdlib.h>
int *p = (int*) malloc(sizeof(int));  // Platz für 1 integer
int *q = (int*) calloc(7, sizeof(int));  // Platz für 7 integers
```

Malloc: keine Initialisierung des Speichers
Calloc: Speicher wird mit 0 initialisiert.

# Explizite Speicheranforderung und -freigabe

- Löschen des zugeteilten Speicherbereichs mit free()

```
free(p); free(q);
```

- Achtung: Pointer *muss* mit malloc/calloc/… erzeugt worden sein. sonst segmentation fault.

Bemerkung: Verwendung von Funktionen zur Speicherzuteilung bzw. -freigabe führt leicht zu Fehlern

Wie das wirklich funktioniert: Systemsoftware im 3. Semester